DATA STRUCTURE LAB

LAB TASK 05



By

MISBAH KHALID

FA25-BCS-005

Submitted to,

Mr. BILAL BUKHARI

October 08, 2026

Department of Computer Sciences

COMSATS University Islamabad,

Attock Campus

LAB TASK 01

Write an application using Stack that checks whether the entered string of brackets is balanced, e.g. [{( )}] is Balanced while {[( )] } is not Balanced because the priority of the pranthesis is not maintained.

CODE:

```
#include <iostream>

#include <string>

using namespace std;


// Node for dynamic stack

struct Node {

    char data;

    Node *next;

};


Node *top = NULL;


// Push a bracket into stack

void push(char value) {

    Node *p = new Node;


    p->data = value;

    p->next = top;

    top = p;

}
```

```
// Remove top element
char pop() {
    if (top == NULL)
        return '\0';

    Node *p = top;
    char value = p->data;

    top = top->next;
    delete p;

    return value;
}

// Check whether brackets match
bool isMatching(char open, char close) {
    if (open == '(' && close == ')')
        return true;

    if (open == '[' && close == ']')
        return true;

    if (open == '{' && close == '}')
        return true;
```

```
    return false;
}

// Check balanced brackets
bool isBalanced(string expression) {

    for (int i = 0; i < expression.length(); i++) {

        char ch = expression[i];

        // Opening brackets
        if (ch == '(' || ch == '[' || ch == '{') {
            push(ch);
        }

        // Closing brackets
        else if (ch == ')' || ch == ']' || ch == '}') {

            // No opening bracket available
            if (top == NULL)
                return false;

            char open = pop();
```

```
            // Brackets do not match
            if (!isMatching(open, ch))
                return false;
        }
    }

    // Stack must be empty
    return top == NULL;
}

int main() {

    string expression;

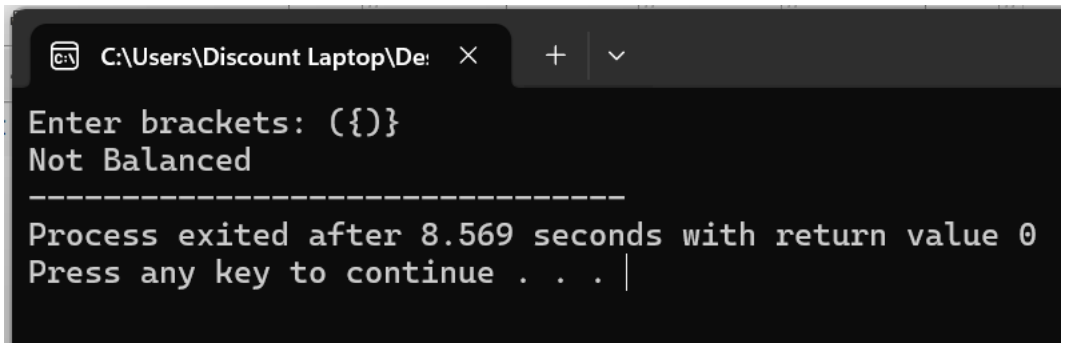
    cout << "Enter brackets: ";
    cin >> expression;

    if (isBalanced(expression))
        cout << "Balanced";
    else
        cout << "Not Balanced";

    return 0;
```

}

OUTPUT:



TASK 02

Use dynamic stack and implement Infix to Postfix conversion algorithm and test it for various inputs.

CODE

```
#include <iostream>

#include <string>

using namespace std;


// Node for dynamic stack

struct Node {

    char data;

    Node *next;

};


Node *top = NULL;


// Push into stack

void push(char value) {

    Node *p = new Node;
```

```
    p->data = value;
    p->next = top;
    top = p;
}

// Pop from stack
char pop() {
    if (top == NULL)
        return '\0';

    Node *p = top;
    char value = p->data;

    top = top->next;
    delete p;

    return value;
}

// Return top element
char peek() {
    if (top == NULL)
        return '\0';
```

```
    return top->data;
}

// Check operator priority
int precedence(char op) {

    if (op == '^')
        return 3;

    if (op == '*' || op == '/')
        return 2;

    if (op == '+' || op == '-')
        return 1;

    return 0;
}

// Check whether character is operator
bool isOperator(char ch) {

    if (ch == '+' || ch == '-' ||
        ch == '*' || ch == '/' || ch == '^')
```

```
        return true;

    return false;
}

// Convert infix to postfix
string infixToPostfix(string infix) {

    string postfix = "";

    for (int i = 0; i < infix.length(); i++) {

        char ch = infix[i];

        // If operand, directly add to postfix
        if (isalnum(ch)) {
            postfix += ch;
        }

        // Opening bracket
        else if (ch == '(') {
            push(ch);
        }
```

```
// Closing bracket
else if (ch == ')') {

    while (top != NULL && peek() != '(') {
        postfix += pop();
    }

    // Remove '('
    if (top != NULL)
        pop();
}

// Operator
else if (isOperator(ch)) {

    while (top != NULL &&
           peek() != '(' &&
           precedence(peek()) >= precedence(ch)) {

        postfix += pop();
    }

    push(ch);
}
```

```
    }

    // Pop remaining operators
    while (top != NULL) {
        postfix += pop();
    }

    return postfix;
}

int main() {

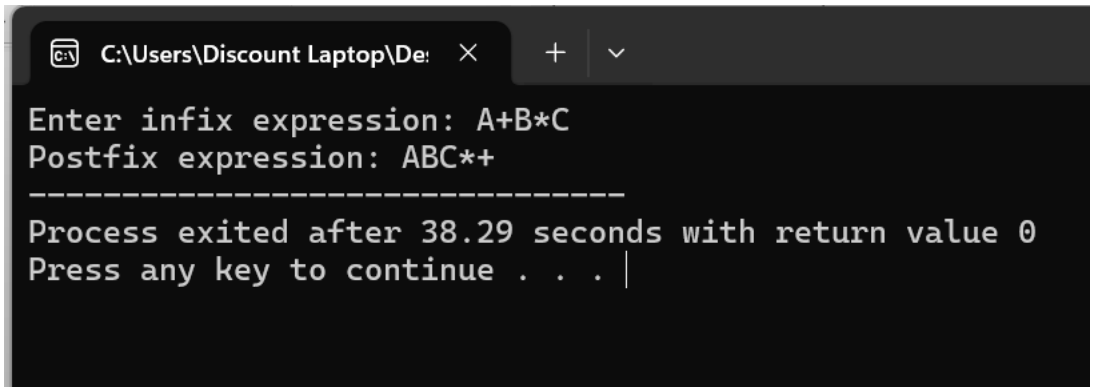
    string infix;

    cout << "Enter infix expression: ";
    cin >> infix;

    cout << "Postfix expression: "
         << infixToPostfix(infix);

    return 0;
}
```

OUTPUT:



TASK 03

For a given postfix expression, use dynamic stack to evaluate a numerical result for given values of variables.

CODE

```
#include <iostream>

#include <string>

#include <cctype>

using namespace std;


// Node for dynamic stack

struct Node {

    int data;

    Node *next;

};


Node *top = NULL;


// Push value into stack
```

```
void push(int value) {

    Node *p = new Node;

    p->data = value;
    p->next = top;

    top = p;
}

// Pop value from stack
int pop() {

    if (top == NULL)
        return 0;

    Node *p = top;

    int value = p->data;

    top = top->next;

    delete p;
```

```
    return value;
}

// Return top value
int peek() {

    if (top == NULL)
        return 0;

    return top->data;
}

// Perform operation
int calculate(int a, int b, char op) {

    if (op == '+')
        return a + b;

    if (op == '-')
        return a - b;

    if (op == '*')
        return a * b;
```

```
    if (op == '/')
        return a / b;

    return 0;
}

// Evaluate postfix expression
int evaluatePostfix(string postfix) {

    int values[26];
    bool entered[26] = {false};

    for (int i = 0; i < postfix.length(); i++) {

        char ch = postfix[i];

        // If operand is a variable
        if (isalpha(ch)) {

            int index = toupper(ch) - 'A';

            // Ask value only once
            if (!entered[index]) {
```

```
            cout << "Enter value of "
                 << char('A' + index) << ": ";

            cin >> values[index];

            entered[index] = true;
        }

        push(values[index]);
    }

    // If operator
    else {

        int b = pop();
        int a = pop();

        int result = calculate(a, b, ch);

        push(result);
    }
}

return pop();
```

```
}

int main() {

    string postfix;

    cout << "Enter postfix expression: ";
    cin >> postfix;

    int result = evaluatePostfix(postfix);

    cout << "Result = " << result;

    return 0;
}
```

OUTPUT:

